

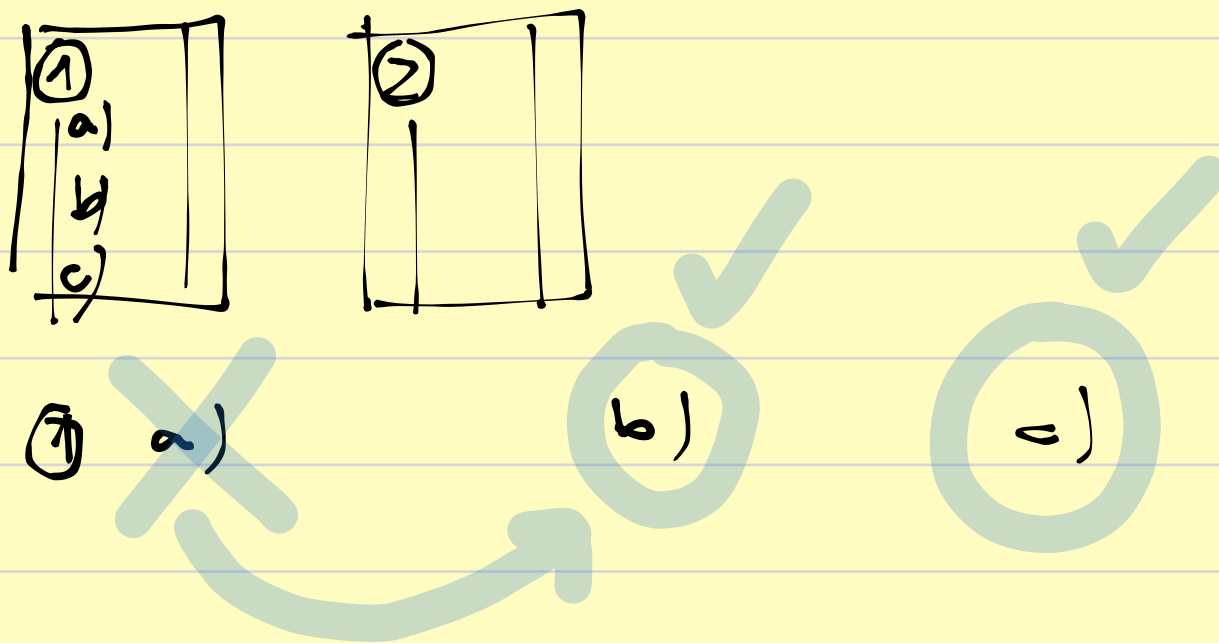
Klausur: Weiterfuehrende KI Programmierung; EDV Nr.:174161; Art de Pruefung: PK
Pruefer: Prof. Dr.-Ing. Dr.-Ing. habil. Javier Villalba-Diez
Name, Vorname: ,
Matrikelnummer:

a	b	c	d	e	f
2	3	6	4	7	1

Datum: 20250704; Ort: Bildungscampus; Bearbeitungszeit: 60 Min.; Zugelassene Hilfsmittel: nicht programmierbarer Taschenrechner. Gesamtpunktzahl: 100 Punkte. Weitere Erlaeuterungen ¹. WICHTIG: ALLE RECHENWEGE MUESSEN DARGELEGT WERDEN. Bitte schreiben Sie nur auf einer Seite des A4-Klausurpapiers

5 uebungen à 20 Punkte → 100 Punkte

¹Versehen Sie alle Boegen mit Ihrem Namen, Matrikelnummer, und Studiengang. Lassen Sie auf jeder Seite einen Seitenrand von 3 cm frei. Verwenden Sie nur dokumentenechte Schreibstifte (keinen Bleistift). ~~Modulpruefungen: Bitte benutzen Sie bei getrennten Pruefern unterschiedliche Klausurbogen.~~ Verwenden Sie nur das bereit gestellte Klausurpapier. Waehrend der Klausur sind keine Fragen zugelassen. Es werden nur lesbare Klausuren bewertet!



👁️ Klausur Sem. WS25/26 ⚡

- 1. Übung (20 Punkte): Gegeben ist ein Datensatz mit zwei Variablen X und Y: $X = \{2a, 5c + 1, 3b, 5d + 1\}$, $Y = \{7a, 2c + 3, 5b, 4d + 4\}$. Bitte ermitteln Sie die lineare Regression und sagen Sie voraus, welchen Wert Y bei $X = 15$ einnimmt.
- 2. Übung (20 Punkte): Gegeben ist ein Grauwertbild I, dessen Pixelwerte sich zyklisch aus den sechs Ziffern Ihrer Ma-

trikelnummer zusammensetzen. $I = \begin{pmatrix} a & b & c & d & e & f \\ b & c & d & e & f & a \\ c & d & e & f & a & b \\ d & e & f & a & b & c \\ e & f & a & b & c & d \\ f & a & b & c & d & e \end{pmatrix}$. Verwenden Sie den folgenden 3×3 -Aufmerksamkeits-Kernel

$K = \begin{pmatrix} 0 & -2 & 0 \\ -2 & 2 & -1 \\ 0 & -1 & 0 \end{pmatrix}$ und führen Sie eine 2-D-Konvolution mit Stride=2,Padding=1 durch.

- 3. Übung (20 Punkte): Die folgenden beiden Kennzahlen-Listen stammen aus vier aufeinander-folgenden Produktionszyklen

Zyklus i	1	2	3	4
Durchlaufzeit	2c+5	2b+3	4d+5	5d+2
Kosten pro Stück	3a+4	5b+4	3c+2	3d+1

Bestimmen Sie den ersten Haupteigenvektor v_1 vom Kennzahlensystem. Wie stark muss die Durchlaufzeit pro Stück (in %) reduziert werden, um eine Kostenreduktion von 10 % zu erreichen?

4. **Aufgabe (20 Punkte):** Der folgende Python-Code soll die Konvolution eines 3x3 Matrix durchführen. Geben Sie die funktionierende Funktion "Define a simple 3x3 kernel (e.g., edge detection)" ein.

```
import numpy as np # For numerical operations
import matplotlib.pyplot as plt # For visualization
import cv2 # OpenCV for image processing (offline use with mock or local image)
```

```
# ----- Step-by-step Convolution on Matrix -----
```

```
# Create a simple 4x4 matrix as input (grayscale image mockup)
```

```
input_matrix = np.array([
    [200, 10, 30, 40],
    [10, 200, 10, 10],
    [10, 10, 110, 120],
    [130, 140, 150, 160]
])
```

```
# Define a simple 3x3 kernel (e.g., edge detection)
```

```
kernel = np.array([
    [-1, -1, -1],
    [-1, 8, -1],
    [-1, -1, -1]
```

```
# Function to apply 3x3 convolution manually (valid padding)
```

A handwriting practice line consisting of a solid top line, a dashed middle line, and a solid bottom line. A thick, black, wavy line is drawn across the middle of the lines, starting from the left margin and extending towards the right. The wavy line oscillates between the top and bottom solid lines, with its peaks and troughs touching the dashed middle line.



```
return output
```

```
# Apply convolution to the sample matrix
output_matrix = apply_convolution(input_matrix, kernel)
```

```
# Visualize input and output
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(input_matrix, cmap='gray')
plt.title("Original_4x4_Matrix")
plt.colorbar()
```

```
plt.subplot(1, 2, 2)
plt.imshow(output_matrix, cmap='gray')
plt.title("After Applying 3x3 Kernel")
plt.colorbar()
plt.show()
```

```
# ----- Convolution on an Image for Edge and Color Detection -----
```

```
# Define kernels
```

```
# Define the kernels
edge_kernel = np.array([
    [-1, -1, -1],
    [-1, 8, -1],
    [-1, -1, -1]
```

11

```
# Red detection kernel (approximation, focus on R channel)
```

```
red_kernel = np.array([
    [0, 0, 0],
    [0, 1, 0],
    [0, 0, 0]
```

11

```
# Blue detection kernel (focus on B channel)
```

```
# Blue detection kernel (focus
blue_kernel = red_kernel.copy()
```

```
# Mockup image as RGB array (4x4 pixels)
```

```
# Mockup image as RGB array (4x4 pixels)
mock_image = np.array([
    [[255, 0, 0], [0, 255, 0], [0, 0, 255], [255, 255, 0]],
    [[255, 0, 0], [255, 0, 0], [0, 0, 255], [0, 255, 255]],
    [[0, 255, 0], [0, 255, 0], [0, 0, 255], [0, 0, 255]],
    [[255, 0, 255], [255, 0, 255], [0, 0, 255], [255, 255, 255]]
], dtype=np.uint8)
```

```
# Function to apply kernel to a single channel
```

```
def convolve_channel(channel, kernel):
    return apply_convolution(channel, kernel)
```

```
# Extract channels
```

```
red_channel = mock_image[:, :, 0]
green_channel = mock_image[:, :, 1]
blue_channel = mock_image[:, :, 2]
```

```
# Apply red and blue detection
```

```
red_detected = convolve_channel(red_channel, red_kernel)
blue_detected = convolve_channel(blue_channel, blue_kernel)
```

```
# Apply edge detection to grayscale conversion of mock image
gray_image = cv2.cvtColor(mock_image, cv2.COLOR_RGB2GRAY)
edge_detected = apply_convolution(gray_image, edge_kernel)
```

5. **Aufgabe (20 Punkte):** Die folgende Python-Funktion soll auf das zweidimensionale Rotkanal vom Bild von Aufgabe 4 "mock image" PCA anwenden.

```
def apply_pca(mock_image):
```

[illegible]

```
return output
```

